

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	Jeeva Raghul .M	Reg. No.:	192524487
Section / Batch:	B Tech AIDS	Date:	02/09/2026

Code of Conduct

I _____ (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

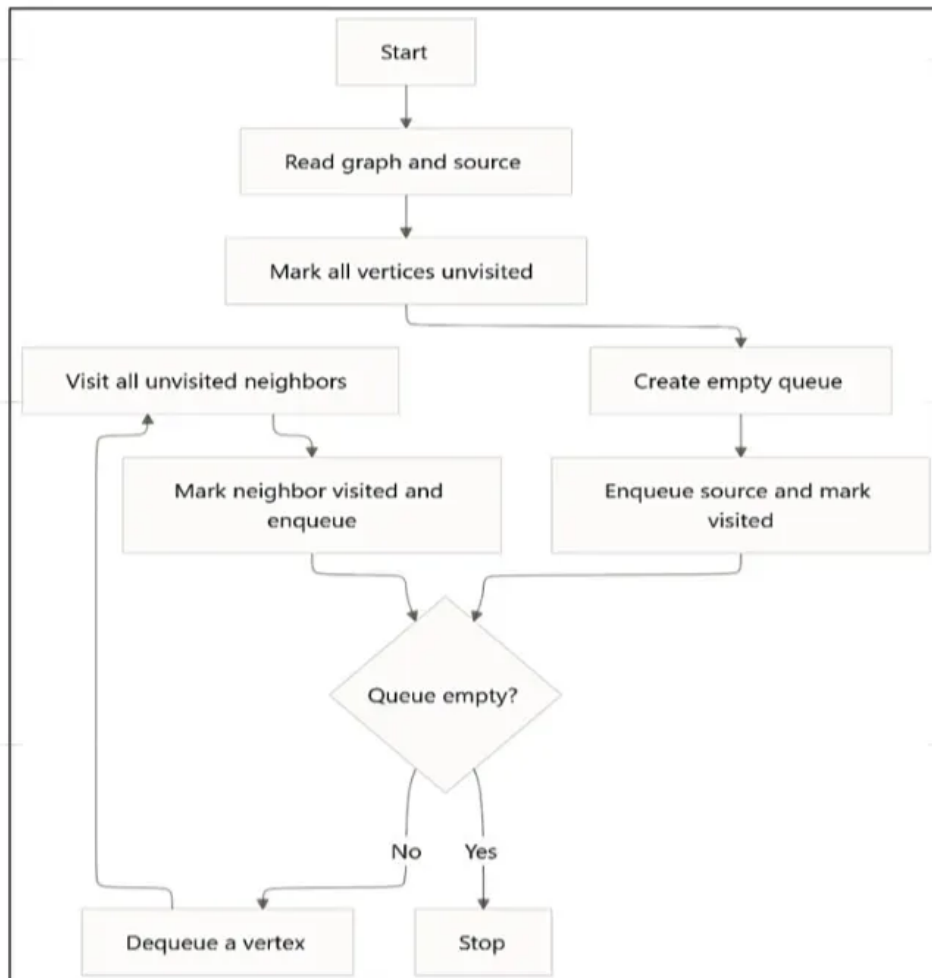
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A – Flowchart-To-Code Conversion

1. Convert the flowchart for Breadth First Search traversal into code.

(20 Marks)

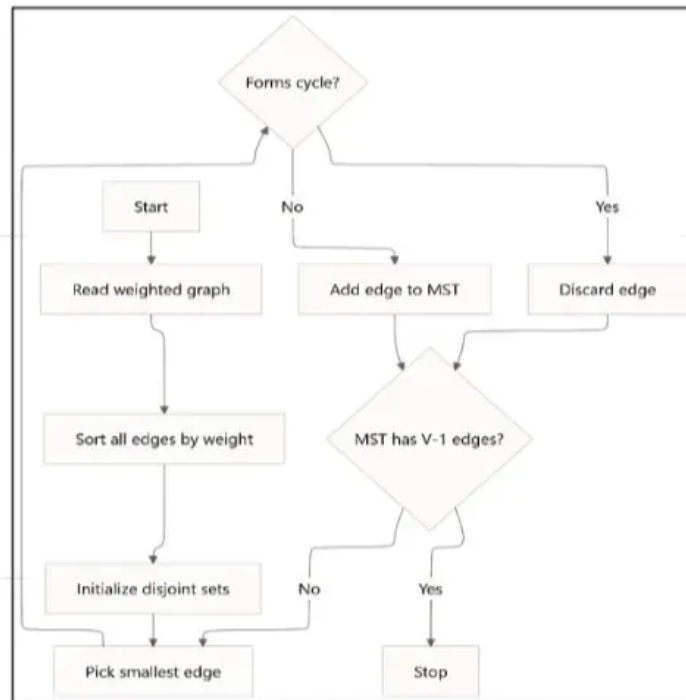
Flowchart Diagram:



2. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

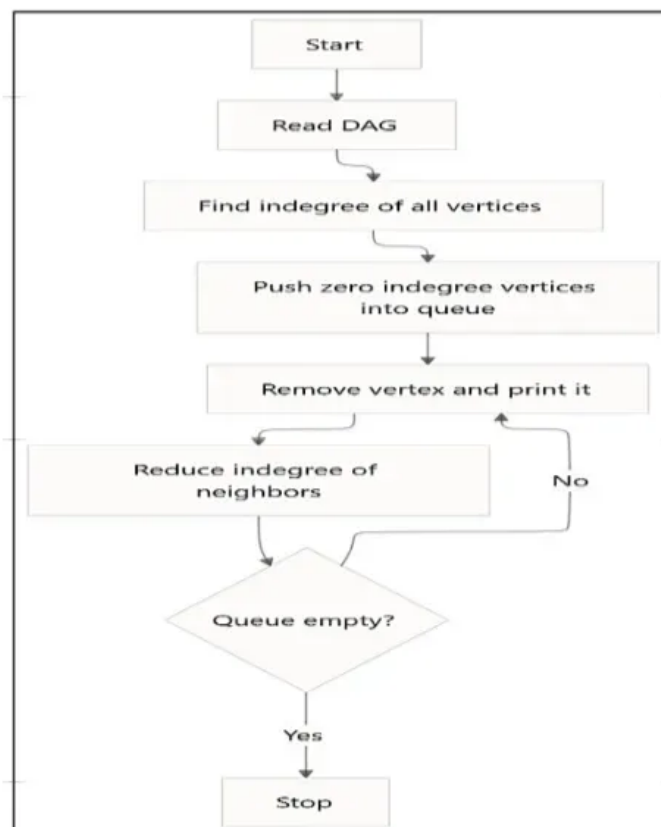
Flowchart Diagram:



3. Convert the flowchart for topological sort into code.

(20 Marks)

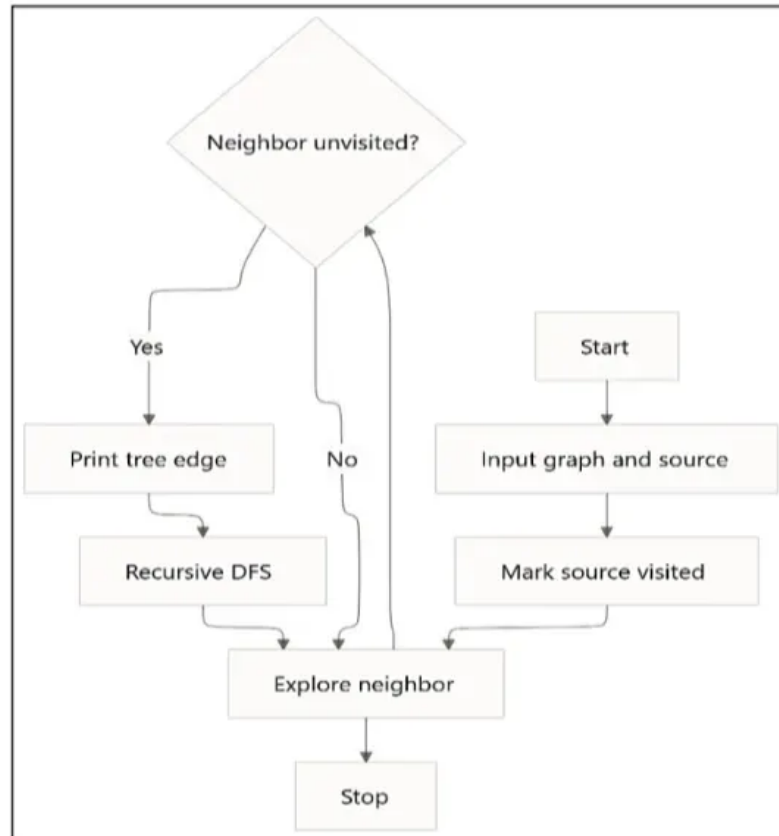
Flowchart Diagram:



4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

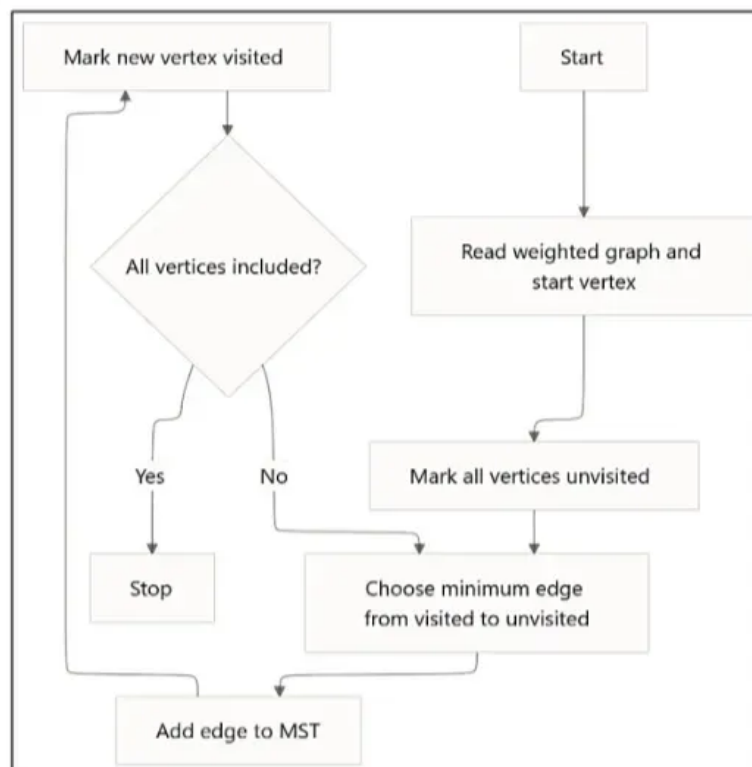
Flowchart Diagram:



5. Convert the flowchart for Prim's algorithm into code.

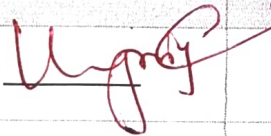
(20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) Breadth First Search:-

```
#include <stdio.h>
```

```
#define MAX 50
```

```
int max()
```

```
{
```

```
    int graph[MAX][MAX], visited[MAX] = {0};
```

```
    int queue[MAX]
```

```
    int n, start, front = 0, rear = 0;
```

```
    int i, v;
```

```
    printf("Enter no. of Vertices: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter adjacency matrix:\n");
```

```
    for (i = 0; i < n; i++)
```

```
        for (int j = 0; j < n; j++)
```

```
            scanf("%d", &graph[i][j]);
```

```
    printf("Enter starting vertex:");
```

```
    scanf("%d", &start);
```

```
    while (front < rear)
```

```
{
```

```
    v = queue[front++];
```

```
    printf("%d", v);
```

```
    for (i = 0; i < n; i++)
```

```
{
```

visited[i] = 1;

queue[rear++] = i;

}

}

}

return 0;

}

2)

koru

2) Kruskal's Algorithm:-

```
#include <stdio.h>
```

```
type of struct {
```

```
int v, u, w;
```

```
} Edge;
```

```
int l[20];
```

```
int find(int x){
```

```
return p[x] == x ? x : p[x] = find(p[x]);
```

```
}
```

```
int main(){
```

```
edge e[30], t;
```

```
int n, m, i, f, c=0;
```

```
scanf("%d %d", &n, &m);
```

```
for (i=0; i<m; i++)
```

```
scanf("%d %d %d", &e[i].v, &e[i].u, &e[i].w);
```

```
for (i=0; i<n; i++) p[i]=i;
```

```
for (i=0; i<m-1; i++)
```

```
for (e[i].w > e[j].w)
```

```
f = e[i], e[i] = e[j], e[j] = f;
```


if (a==b)

{

 P[a]=b;

 c++;

 }

}

return;

}

code:

#include

code:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, a[10], inc[10] = {0}, ac[10], k=0, x=0;
```

```
scanf ("%d", &n);
```

```
for (i=0; i<n; i++)
```

```
for (j=0; j<n; j++) {
```

```
scanf ("%d", &a[i][j]);
```

```
if (a[i][j]) inc[j]++;
```

```
}
```

```
for (i=0; i<n; i++)
```

```
if (inc[i]==0) a[x++] = i;
```

```
while (x<n) {
```

```
printf ("%d", x);
```

```
for (i=0; i<n; i++)
```

```
if (inc[i]==0)
```

```
· a[x++] = i
```

```
}
```

```
return 0;
```

```
}
```

4) code:

```
#include <stdio.h>
```

```
int a[10][10], v[10], n;
```

```
void dfs(int x){
```

```
    int i;
```

```
    v[x] = 1;
```

```
    for (i=0; i<n; i++){
```

```
        if (a[x][i] == 1 & v[i] == 0)
```

```
            printf("%d %d\n", x, i);
```

```
            dfs(i);
```

```
    }
```

```
}
```

```
int main()
```

```
int s, i, j, t;
```

```
scanf("%d", &n);
```

```
for (i=0; i<n; i++){
```

```
    scanf("%d", &a[i][j]);
```

```
    scanf("%d", &s);
```

```
    dfs(s);
```

```
}
```

```
return 0;
```

```
}
```

1. Introduction

2. Objectives

3. Scope of the Study

4. Methodology

5. Results and Discussion

6. Conclusion

7. References

8. Appendix

9. Index

10. Summary

11. Abstract

12. Keywords

13. Notes

14. References

15. Appendix

16. Index

17. Summary

18. Abstract

19. Keywords

20. Notes

21. References

22. Appendix

23. Index

24. Summary

6) Prim's Algorithm:-

code:

```
#include <stdio.h>
```

```
#define RVF9999
```

```
int main()
```

```
{
```

```
int n, a[103][103], v[103] = {0};
```

```
scanf ("%d", &n);
```

```
for (i=0; i<n; i++)
```

```
for (j=0; j<n; j++)
```

```
scanf ("%d", &a[i][j]);
```

```
scanf ("%d", &v[0]);
```

```
v[0] = 1;
```

```
for (k=1; k<n; k++) {
```

```
min = RVF9999;
```

```
for (i=0; i<n; i++)
```

```
if (v[i])
```

```
for (j=0; j<n; j++)
```

```
if (v[j] == 0 & a[i][j] < min)
```

```
min = a[i][j], u=i, w=j;
```

```
printf ("%d - %d ; %d\n", u, w, min);
```

```
v[w] = 1;
```

```
}
```

```
return 0;
```